University of Colorado Denver

Department of Computer Science & Engineering

CSCI/CSCY 4407: Security & Cryptography

Lab 10 Assignment: Cryptographic Commitment Schemes, Fair Protocols, and Secure Summation

Instructor: Dr. Victor Kebande

Teaching Assistant: Celest Kester

Spring 2026

---

1 Assignment Instructions

This is a group-based lab assignment. Each group must complete and submit its own original work. You may discuss high-level ideas related to commitment schemes, fairness, hiding, binding, cryptographic hashing, symmetric encryption, public-key encryption, protocol design, and secure multiparty computation. However, you are not permitted to share Python scripts, screenshots, outputs, attack traces, written explanations, or final report text with other groups.

This lab combines both implementation and analysis. Some tasks require you to implement commitment schemes and protocols in Python. Other tasks require you to reason carefully about why a protocol fails, why a construction satisfies or fails a security property, and how the implementation results support or contradict the intuition. Several tasks explicitly require **pen-and-paper reasoning**. In those tasks, you are expected to show intermediate steps clearly and not merely state final answers.
Each group must generate and use its own:

- working directory,
- Python files,
- random values,

- outputs and screenshots,
- written interpretations and conclusions.

Because each group will use different random seeds, messages, keys, and experimental trials, exact outputs will differ. This is intentional and forms part of the assessment design. All submitted work must reflect your group's own implementation, screenshots, analysis, and explanations.

2 Academic Integrity and Collaboration

- Allowed: discussing lecture concepts and high-level cryptographic ideas.
- Allowed: consulting Python documentation and standard library documentation.
- Not allowed: sharing code, screenshots, outputs, reports, or answers with other groups.
- Not allowed: copying solutions from the internet, public repositories, generative tools, or previous student work without independent implementation and interpretation.
- Submission: only one group member should submit the final report on Canvas on behalf of the group.

3 Introduction

In many cryptographic systems, secrecy alone is not enough. There are situations in which one party must choose a value now, keep it hidden temporarily, and reveal it later without being allowed to change it. This tension arises naturally in remote gambling, secure voting, distributed protocols, zero-knowledge systems, auctions, and secure multiparty computation. The central problem is not simply whether a value can be hidden, but whether it can be hidden *and fixed* at the same time.

A cryptographic commitment scheme solves exactly this problem. Informally, a commitment scheme allows a sender to lock a value in a cryptographic container during a *commit phase*, and later reveal both the value and the opening information during an *open phase*. The receiver should learn nothing about the committed value before the opening phase, yet should be confident that the sender cannot open the same commitment to two different values. The lecture emphasized this "commit now, reveal later" view and summarize the two essential properties as *hiding* and *binding*.

More formally, a commitment scheme is presented as a triple of algorithms (P, C, V), where parameter generation outputs public parameters, the commitment algorithm produces a commitment and decommitment information, and the verification algorithm checks whether an opening is valid. This was explicitly presented and this structure showed how a commitment can be viewed as the cryptographic analogue of placing a value into a locked safe and later revealing the key.

The motivating example in the slides is the Internet casino. In the naive protocol, the player sends a guess G, and the casino then chooses a value T. If $G = T$, the player wins. But because the casino sees G first, it can always choose $T \neq G$, making the protocol fundamentally unfair. It was then showed how commitment fixes this by forcing the party that chooses first to lock in its choice before learning the other party's input.

In this lab you will implement and evaluate these ideas directly. You will first analyze why the unfair casino protocol fails. You will then implement several commitment constructions in Python, including weak and stronger hash-based variants. You will empirically test hiding and binding behavior under controlled message spaces. You will then use commitment schemes to implement a fair two-party coin-flipping protocol, and finally extend the discussion to secure summation, where multiple parties want to compute a sum without revealing their individual inputs.

The goal of this lab is not only to make the code run, but to make you understand why the mechanisms are needed, what security properties they are supposed to provide, how weak constructions fail, and how cryptographic primitives become building blocks for larger protocols.

4 Motivating Case Scenario

Case Scenario: A Remote Gambling Protocol That Looks Fair but Is Not

A player and an online casino want to run a simple game remotely. The player sends a guess $G \in \{1, 2, \ldots, 100\}$ and pays \$1 to play. The casino then chooses a number $T \in \{1, 2, \ldots, 100\}$. If $G = T$, the player wins \$200. Otherwise, the player wins \$0. At first glance, this seems like a reasonable remote gambling protocol.

However, the order of information completely changes the security of the protocol. If the casino learns G before choosing T, it can always choose $T \neq G$, ensuring that the player never wins. The lecture slides use this example to show that remote fairness fails if one party can adapt after learning the other party's input. They then introduce a commitment-based repair in which the first mover locks its choice before the other party responds.

This lab uses that example as the starting point for everything that follows. You will first analyze the unfair protocol, then implement commitment schemes, then replace the unsafe interaction with a commitment-based fair protocol, and finally connect these ideas to broader protocol design.

5 Assignment Objectives

By completing this lab, you will be able to:

- Explain why protocol fairness can fail even when randomness is involved.

- Explain the role of commitment schemes in preventing adaptive cheating.
- Distinguish clearly between hiding and binding.
- Implement multiple commitment constructions in Python.
- Empirically test whether a construction appears hiding or fails hiding in a restricted message space.
- Reason about when a construction is or is not binding.
- Implement a fair two-party coin-flipping protocol using commitments.
- Explain why commitment schemes are useful in protocol design beyond gambling examples.
- Relate commitment ideas to secure summation and secure multiparty computation.

6 Overview of Lab Tasks

Task	**Title**	**Primary Learning Goal**
Task 1	Working Directory and Setup	Prepare the environment and create baseline files for experiments.
Task 2	Unfair Casino Protocol Analysis	Understand why the naive protocol is unfair and compute the player's expected outcome.
Task 3	Commitment Scheme Interface in Python	Build a reusable Python framework for commit, open, and verify.
Task 4	Weak Hash Commitment Attack	Show why deterministic hashing is not hiding over a small message space.
Task 5	Randomized Hash Commitment	Implement a stronger construction and test its behavior.
Task 6	Hiding Experiments and Adversary Reasoning	Evaluate how hiding changes when the message space is small versus salted.
Task 7	Binding Analysis and Collision Reasoning	Explain what binding means and why collision resistance matters.
Task 8	Encryption-Based Commitments	Implement and compare symmetric- and public-key-based commitment ideas.
Task 9	Fair Coin-Flipping Protocol	Use a commitment scheme to implement a fair two-party protocol.
Task 10	Protocol Security Reasoning	Analyze who can cheat, when, and why.
Task 11	Secure Summation Implementation	Implement a small secure summation protocol in Python.
Task 12	Final Reflection and Comparison	Connect all lab components and reflect on protocol design.

7 Lab Requirements

To complete this lab successfully, students must prepare a Python-based environment suitable for hashing, randomness generation, and protocol simulation. The recommended platforms are:

- Ubuntu Linux
- Kali Linux
- A workstation running Linux
- A Linux virtual machine using VirtualBox or VMware

7.1 Required Software Tools

The following tools are recommended:

- Python 3
- Python standard library: `hashlib`, `secrets`, `random`, `itertools`, `json`
- Linux terminal
- Text editor or code editor such as nano, vim, gedit, or VS Code

7.2 Required Installations

Install Python if it is not already available:

```
sudo apt update
sudo apt install python3 python3-pip
```

Verify the installation:

```
python3 --version
```

7.3 Working Directory Setup

Create a dedicated working folder for this lab:

```
mkdir Commitment_Schemes_Lab
cd Commitment_Schemes_Lab
```

Recommended subfiles and scripts:

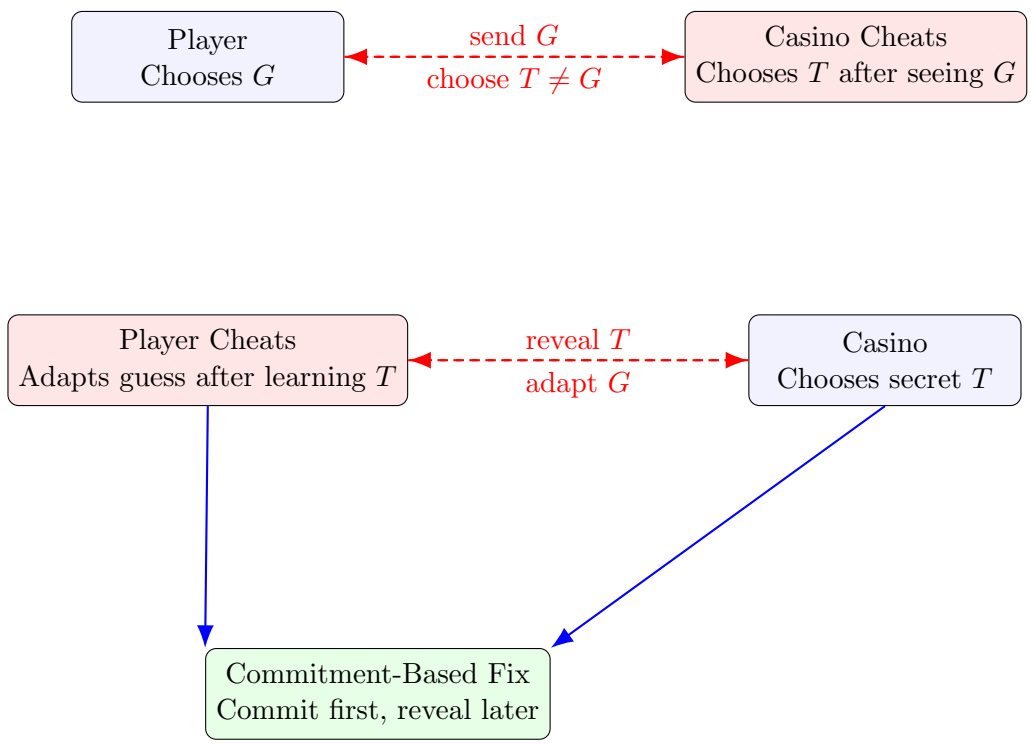
- `casino_naive.py`
- `commitment_utils.py`
- `weak_hash_attack.py`
- `coinflip.py`
- `secure_summation.py`

7.4 Documentation Requirements

Your final report must include:

- screenshots showing executed Python commands and outputs,
- evidence of successful and unsuccessful verification cases,
- evidence of brute-force recovery in the weak commitment task,
- explanations of what each script is doing,
- pen-and-paper reasoning where explicitly required,
- a final comparison and reflection.

8 Conceptual Lab Diagram



Interpretation: The top interaction shows how the casino cheats if it can wait to see the player's guess before choosing its own value. The middle interaction shows the reverse problem: if the player can delay its choice until after learning the casino's value, the player can also cheat. The bottom box shows the cryptographic repair: commitment forces a party to fix its value first while keeping it hidden until the reveal phase. This prevents either side from adapting after learning the other party's input.

9 Lab Tasks

This lab is completed step by step. Read each task carefully before implementation. Do not skip ahead. Several later tasks depend on files created in earlier tasks.

9.1 Task 1: Create the Working Directory and Baseline Files

Begin by creating the working directory and preparing a few small files that will be used as messages or protocol inputs. These files are not yet the protocol themselves, but they provide concrete values that make later experiments easier to interpret.
Step 1: Create the directory

```
mkdir Commitment_Schemes_Lab
cd Commitment_Schemes_Lab
```

Step 2: Create a few simple text files

```
echo "42" > guess_player.txt
echo "77" > casino_secret.txt
echo "1" > coin_input_a.txt
echo "0" > coin_input_b.txt
```

Step 3: Display the file contents

```
cat guess_player.txt
cat casino_secret.txt
cat coin_input_a.txt
cat coin_input_b.txt
```

Step 4: Inspect the working directory

```
pwd
ls -l
```

Step 5: Compute SHA-256 hashes of the plaintext files

```
sha256sum guess_player.txt casino_secret.txt coin_input_a.txt coin_input_b.txt
```

What to do:

- Create the working directory.
- Create the four files.
- Display the contents.
- Inspect the directory contents.
- Compute SHA-256 hashes for all files.

- Include screenshots showing the setup and file creation.

What to explain:

- Why it is useful to have concrete values before moving into commitment experiments.
- Why a protocol example becomes easier to reason about when the values are fixed and visible.

9.2 Task 2: Analyze the Unfair Casino Protocol (Pen-and-Paper Only)

This task must be completed entirely using pen and paper. You may type the final write-up into your report, but your reasoning and derivation must be shown clearly.
The lecture slides introduce an Internet casino protocol in which the player sends a guess G and the casino then chooses T. They explicitly note that the casino can cheat by choosing $T \neq G$ after seeing the player's guess.
Assume the following naive protocol:

- The player pays \$1 and sends $G \in \{1, \ldots, 100\}$.
- The casino chooses $T \in \{1, \ldots, 100\}$.
- If $G = T$, the player receives \$200.
- Otherwise, the player receives \$0.

Step 1: Write the protocol as a sequence of formal steps.
Step 2: Compute the expected payout *if* the casino is honest and chooses T uniformly at random.
Step 3: Describe how the casino cheats if it sees G before choosing T.
Step 4: Explain what happens to the player's expected value once the casino adopts that strategy.
What to do:

- Write a short paragraph describing the honest protocol.
- Compute the expected payout under honest play.
- Write a short paragraph describing the cheating strategy.
- Compute or explain the player's expected payout under the cheating strategy.

What to explain:

- Why the protocol is unfair even though randomness appears to be involved.
- Why the problem is not weak randomness, but the order of information flow.

9.3 Task 3: Build a Python Commitment Interface

You will now create a reusable commitment framework in Python. The purpose is not merely to make one example work, but to define a consistent interface for multiple commitment constructions.

Step 1: Create the script

```
nano commitment_utils.py
```

Step 2: Implement the following structure:

```
import hashlib
import secrets

def sha256_bytes(data: bytes) -> str:
    return hashlib.sha256(data).hexdigest()

def commit_hash_deterministic(message: str):
    """
    Weak scheme:
    c = H(m)
    opening information is just m
    """
    c = sha256_bytes(message.encode())
    return c, {"message": message}

def verify_hash_deterministic(c: str, opening: dict) -> bool:
    message = opening["message"]
    return sha256_bytes(message.encode()) == c

def commit_hash_randomized(message: str):
    """
    Better hash-based commitment:
    r <- {0,1}^128
    c = H(r || m)
    opening information is (r, m)
    """
    r = secrets.token_bytes(16)
    c = sha256_bytes(r + message.encode())
    return c, {"message": message, "randomness_hex": r.hex()}

def verify_hash_randomized(c: str, opening: dict) -> bool:
    message = opening["message"]
    r = bytes.fromhex(opening["randomness_hex"])
    return sha256_bytes(r + message.encode()) == c
```

Step 3: Add a small test block:

```
if __name__ == "__main__":
    m = "42"
    c1, o1 = commit_hash_deterministic(m)
    print("Weak commitment:", c1)
    print("Weak verify:", verify_hash_deterministic(c1, o1))

    c2, o2 = commit_hash_randomized(m)
    print("Randomized commitment:", c2)
    print("Randomized verify:", verify_hash_randomized(c2, o2))
```

Step 4: Run the script

```
python3 commitment_utils.py
```

What to do:

- Implement both commitment variants.
- Test that both verification routines accept correct openings.
- Include screenshots of the file and output.

What to explain:

- Why both scripts appear to "work" at a functional level.
- Why functional correctness is different from security.

9.4 Task 4: Attack the Weak Hash Commitment

The task discuss a hash-based construction that is binding if the hash is collision-resistant, but not hiding in a small message space such as the casino example where $M = G \in \{1, \ldots, 100\}$. They explicitly show that the casino can try all values and recover the committed message.
You will now reproduce that attack experimentally.
Step 1: Create the script

```
nano weak_hash_attack.py
```

Step 2: Use the deterministic scheme from Task 3 and choose a secret message in the range 1 to 100.
Step 3: Write a brute-force attacker that loops through all candidates and checks which one hashes to the observed commitment.
A possible structure is:

```
from commitment_utils import commit_hash_deterministic
import hashlib

secret_message = "77"
```

```
c, opening = commit_hash_deterministic(secret_message)

print("Observed commitment:", c)

recovered = None
for i in range(1, 101):
    guess = str(i)
    if hashlib.sha256(guess.encode()).hexdigest() == c:
        recovered = guess
        break

print("Recovered message:", recovered)
```

Step 4: Run the script

```
python3 weak_hash_attack.py
```

Step 5: Repeat the experiment with a few different secrets.
What to do:

- Implement the brute-force attacker.
- Recover the message for several commitments.
- Include screenshots of successful recoveries.

What to explain:

- Why the scheme fails hiding.
- Why this attack is feasible in small domains.
- How this directly connects to the unfair Internet casino example in the slides.

9.5 Task 5: Test the Randomized Hash Commitment

The task then present a stronger hash-based commitment in which a random string K is sampled and the commitment is formed as $C = H(K\|M)$. This is presented as a better construction because the randomizer hides the message even when the message space is small. You will now test that behavior experimentally.
Step 1: Create the script

```
nano randomized_hash_experiment.py
```

Step 2: Commit to the same message five times using the randomized scheme.
Step 3: Print all commitments and the corresponding opening information.
Step 4: Verify each one.
A useful skeleton:

```
from commitment_utils import commit_hash_randomized, verify_hash_randomized

m = "42"
results = []

for _ in range(5):
    c, opening = commit_hash_randomized(m)
    ok = verify_hash_randomized(c, opening)
    results.append((c, opening, ok))

for idx, item in enumerate(results):
    print(f"Trial {idx+1}")
    print("Commitment:", item[0])
    print("Opening:", item[1])
    print("Verify:", item[2])
    print("-" * 40)
```

Step 5: Run the script

```
python3 randomized_hash_experiment.py
```

What to do:

- Run the experiment.
- Show that the same message produces different commitments.
- Verify that each opening still checks correctly.

What to explain:

- Why the same message does not always produce the same commitment.
- Why the randomizer helps hiding.
- Why this does not weaken verification.

9.6 Task 6: Empirical Hiding Experiment With a Guessing Adversary

This task asks you to experimentally model a simple hiding adversary.
Step 1: Choose two messages from a small domain, for example $m_0 = "17"$ and $m_1 = "42"$.
Step 2: Write a script that randomly commits to one of them under the deterministic scheme, then lets an attacker guess which one was used.
Step 3: Repeat the experiment under the randomized scheme.
A minimal structure is:

```
import random
from commitment_utils import commit_hash_deterministic, commit_hash_randomized

m0 = "17"
m1 = "42"

def attacker_deterministic(c):
    # perfect dictionary attack in tiny space
    import hashlib
    if hashlib.sha256(m0.encode()).hexdigest() == c:
        return 0
    return 1

def run_trials(commit_func, attacker, trials=100):
    wins = 0
    for _ in range(trials):
        b = random.randint(0,1)
        m = m0 if b == 0 else m1
        c, _ = commit_func(m)
        guess = attacker(c)
        if guess == b:
            wins += 1
    return wins / trials
```

For the randomized version, design a simple attacker and evaluate whether it performs close to random guessing.

What to do:

- Implement the experiment.
- Report the empirical guessing accuracy for both schemes.
- Use at least 100 trials.

What to explain:

- Why the deterministic scheme is easy to distinguish.
- Why the randomized scheme should push the adversary closer to guessing.
- How this connects conceptually to the hiding game shown in the slides. index=10

9.7 Task 7: Binding Analysis (Pen-and-Paper)

The task formally define binding by asking whether an adversary can produce one commitment C and two different valid openings (M_0, K_0) and (M_1, K_1) with $M_0 \neq M_1$.
This task must be completed with pen-and-paper reasoning.

Step 1: State, in your own words, what it would mean to break binding.
Step 2: Explain why collision resistance supports binding for the deterministic hash commitment $C = H(M)$.
Step 3: Explain why binding in the randomized hash commitment $C = H(r\|M)$ would require finding two different pairs $(r_0, M_0) \neq (r_1, M_1)$ with the same hash output.
Step 4: Explain why proving full cryptographic binding is stronger than merely failing to find a collision in a few experiments.
What to do:

- Write a careful explanation of binding in both constructions.
- Include at least one formal expression.

What to explain:

- Why "I tried and failed" is not a proof of security.
- Why computational infeasibility is the right lens.

9.8 Task 8: Implement Commitment Ideas From Encryption

The task focuses on the commitment schemes built from symmetric encryption and from public-key encryption. They note that an encryption-based construction may be hiding, but binding must be argued carefully, and that public-key based commitment can be binding relative to the public key because the ciphertext has a unique decryption under the secret key.
For this task, you will implement *toy* commitment ideas based on XOR-style encryption and RSA-OAEP using a Python library if available. If you cannot use a crypto library, implement only the symmetric toy version and explain the public-key version conceptually.
Part A: Toy symmetric construction
Step 1: Create

```
nano toy_symmetric_commit.py
```

Step 2: Implement a toy "encrypt" function using XOR of a repeated key stream over short ASCII messages. This is for education only and not real security.
Step 3: Define:

$$C = Enc_K(M)$$

and use (M, K) as the opening information.
Step 4: Verify by decrypting and checking equality.
Part B: Conceptual public-key construction
If you have a Python crypto library available, implement a public-key encryption based commitment. If not, explain in your report how:

$$(pk, sk) \leftarrow \mathcal{K}, \quad C \leftarrow E_{pk}(M; R)$$

can serve as a commitment, and why opening consists of revealing (M, R).
What to do:

- Implement the toy symmetric version.
- Show that the verification logic works.
- Explain the public-key version either experimentally or conceptually.

What to explain:

- Why hiding appears plausible in the encryption-based view.
- Why a toy encryption experiment is not itself a proof of commitment security.

9.9 Task 9: Implement the Coin-Flipping Protocol

The task present a commitment-based common coin protocol in which Alice chooses a bit a, commits to it, Bob chooses b, Alice reveals, and the output is $c = a \oplus b$. They emphasize that neither party should be able to control the final coin if the commitment scheme is secure. You will now implement that protocol in Python.
Step 1: Create the script

```
nano coinflip.py
```

Step 2: Use the randomized hash commitment from Task 3.
Step 3: Implement the following flow:

- Alice chooses random bit $a \in \{0, 1\}$.
- Alice computes $(C, opening) \leftarrow Commit(a)$.
- Bob chooses random bit $b \in \{0, 1\}$.
- Alice reveals a and the opening information.
- Bob verifies and computes $c = a \oplus b$.

A useful skeleton:

```
import random
from commitment_utils import commit_hash_randomized, verify_hash_randomized

a = str(random.randint(0,1))
C, opening = commit_hash_randomized(a)
b = random.randint(0,1)

print("Alice commitment:", C)
print("Bob bit:", b)
print("Alice opens with:", opening)

ok = verify_hash_randomized(C, opening)
if ok:
```

```
    c = int(opening["message"]) ^ b
    print("Common coin:", c)
else:
    print("Verification failed")
```

Step 4: Run the script many times.
What to do:

- Implement the protocol.
- Run at least 20 trials.
- Record outputs and show that both parties can compute the same coin if verification succeeds.

What to explain:

- Why the commitment must be sent before Bob chooses b.
- Why $a \oplus b$ acts as the final common coin.

9.10 Task 10: Analyze Cheating in the Coin-Flipping Protocol (Pen-and-Paper)

This is one of the most important tasks in the lab.
Step 1: Explain what Alice could try to do if there were no binding property.
Step 2: Explain what Bob could try to do if the commitment leaked a before the opening phase.
Step 3: Explain why hiding protects against Bob's cheating and binding protects against Alice's cheating.
Step 4: Explain what changes if Alice uses the weak deterministic hash commitment over a tiny message space.
What to do:

- Write a step-by-step adversarial analysis.
- Explicitly identify which property protects against which adversary.

What to explain:

- Why fair coin flipping needs both hiding and binding.
- Why one property alone is not enough.

9.11 Task 11: Implement Secure Summation in Python

The task now moves to secure summation, where several parties want to compute $x_1 + \cdots + x_n$ without revealing the individual x_i. They present a secret-sharing style protocol using random shares and column sums.
You will implement a small version for $n = 3$ parties.
Assume:

$$x_1, x_2, x_3 \in \mathbb{Z}_N$$

for a chosen modulus $M = 3N$.
Step 1: Create the script

```
nano secure_summation.py
```

Step 2: Let each party i split x_i into three random shares $x_{i,1}, x_{i,2}, x_{i,3}$ such that:

$$x_{i,1} + x_{i,2} + x_{i,3} \equiv x_i \pmod{M}.$$

Step 3: Form column sums:

$$C_j = x_{1,j} + x_{2,j} + x_{3,j} \pmod{M}$$

Step 4: Let each party recover the total sum from the column sums.
A simple coding approach is:

```
import random

N = 100
M = 3 * N
inputs = [17, 42, 23]

def share_value(x, num_parties=3, mod=M):
    shares = [random.randint(0, mod-1) for _ in range(num_parties-1)]
    final_share = (x - sum(shares)) % mod
    shares.append(final_share)
    return shares

matrix = [share_value(x) for x in inputs]
column_sums = [sum(matrix[i][j] for i in range(3)) % M for j in range(3)]
total = sum(column_sums) % M

print("Inputs:", inputs)
print("Share matrix:", matrix)
print("Column sums:", column_sums)
print("Recovered total:", total)
print("Actual total mod M:", sum(inputs) % M)
```

Step 5: Run with different inputs.
What to do:

- Implement the protocol.
- Run the experiment several times.
- Show that the recovered sum matches the true sum modulo M.

What to explain:

- Why individual shares do not reveal the private input directly.
- Why the final sum is still recovered correctly.
- How this connects to the secure summation slides.

9.11 Task 12: Final Reflection and Comparison

Your final reflection should address all of the following:

- Why the naive casino protocol is unfair.
- Why commitment schemes repair that fairness problem.
- Why deterministic hashing can fail hiding in small domains.
- Why adding randomness changes the security picture.
- Why hiding and binding are different and complementary.
- How coin flipping uses commitment to enforce fairness.
- How secure summation extends the same mindset toward secure multiparty computation.

You must also include one short comparison table contrasting:

- naive casino protocol,
- weak deterministic hash commitment,
- randomized hash commitment,
- commitment-based coin-flipping protocol.

10 Submission Requirements

- Submit a single PDF report on Canvas by the stated deadline. Only one group member should submit on behalf of the group.
- Your report must be clearly structured and organized by task.
- For each task, your report must include:
  - a clear explanation of what was implemented or analyzed,
  - screenshots showing terminal execution where applicable,
  - Python code screenshots or listings where applicable,
  - interpretation of the results, not just raw output.
- Your Python source code must be included either as embedded code in the report or as separate files if instructed by the instructor.
- All results must be reproducible. Missing screenshots, unclear code, or undocumented outputs may result in reduced points.

11 Task-Specific Requirements

- Task 1: Evidence of directory creation, file creation, file display, and hashes.
- Task 2: Clear written analysis of the unfair casino protocol and the player's expected value.
- Task 3: Correct implementation of the weak and randomized hash commitment interfaces.
- Task 4: Evidence of successful brute-force recovery against the weak scheme.
- Task 5: Evidence that repeated commitments to the same message differ under the randomized scheme.
- Task 6: Guessing experiment comparing weak and randomized schemes.
- Task 7: Careful binding discussion with formal reasoning.
- Task 8: Correct toy encryption-based implementation and conceptual analysis.
- Task 9: Python script and outputs demonstrating fair commitment-based coin flipping.
- Task 10: Step-by-step security analysis identifying the roles of hiding and binding.
- Task 11: Python script and outputs demonstrating correct secure summation.
- Task 12: Comparison table and final reflection.

12 Grading Rubric (Total: 100 Points)

Assessment Criteria	**Points**
Correct working directory setup, baseline files, and hash evidence	6
Clear pen-and-paper analysis of the unfair casino protocol, including expected-value reasoning	10
Correct Python implementation of the commitment interface with both weak and randomized hash schemes	10
Successful brute-force attack against the weak deterministic commitment and clear explanation of why hiding fails	10
Careful experimental analysis of the randomized commitment and explanation of why repeated commitments differ	8
Hiding experiment with empirical adversary reasoning and comparison between weak and randomized schemes	8
Binding analysis with correct formal reasoning and discussion of collision resistance	10
Encryption-based commitment implementation or strong conceptual analysis and comparison to hash-based commitments	8
Correct implementation of the fair coin-flipping protocol and clear explanation of why both parties are protected	10
Pen-and-paper protocol security analysis identifying how hiding and binding prevent cheating	8
Correct implementation of secure summation and explanation of why private inputs remain protected	6
Report quality: clarity, completeness, screenshots, organization, reproducibility, and depth of explanation	6
Total	**100**

13 Notes to Students

This lab contains both practical code-based experiments and conceptual reasoning. Some tasks use Python implementations of commitment ideas and protocols, while other tasks require pen-and-paper analysis to explain why a protocol succeeds or fails. The toy construc-

tions and experiments are educational and are not meant for direct real-world deployment. Your report should clearly distinguish between:

- what is being implemented for learning purposes,
- what is simplified relative to real cryptographic practice,
- what security property is being tested,
- and what assumption is required for that property to hold.